

INSTITUTO TECNOLÓGICO DE COSTA RICA  
Campus Tecnológico Central Cartago  
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras  
II Semestre 2024

Reporte #6 - Semana 7 – Sivarama - Capítulo 10  
Más sobre el lenguaje ensamblador

**Estudiante:** Mauricio González Prendas – **Carné:** 2024143009

**Profesor:** Esteban Arias Méndez

**Fecha de entrega:** 09/09/2024

### Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 211–229.

**Abstract**—This paper provides an overview of essential assembly language constructs, including instructions, directives, and macros. It explains key instructions like XCHG for data exchange and XLAT for byte translation, as well as shift and rotate operations such as SHL and ROL. The paper also covers constant definitions using directives like EQU, %assign, and %define, and explores macros with and without parameters, showcasing their use in data transfer and arithmetic operations. These elements are fundamental for efficient low-level programming and code optimization.

---

## I. Introducción a lenguaje ensamblador

Los programas en lenguaje ensamblador se componen de tres tipos de instrucciones:

1. **Instrucciones ejecutables (opcodes):** Le dicen al procesador qué hacer y generan instrucciones en lenguaje máquina.
2. **Directivas del ensamblador (pseudo-ops):** No son ejecutables y le proporcionan información al ensamblador.
3. **Macros:** Proveen una sustitución de texto avanzada y se detallan en otro capítulo.

Las instrucciones en lenguaje ensamblador se escriben en líneas con un formato común: [etiqueta] mnemonico [operandos] [/comentario]. Aunque los espacios no afectan al ensamblador, son útiles para mejorar la legibilidad.

### Asignación de Datos

En lenguajes de alto nivel como C, la asignación de espacio de memoria depende de los tipos de datos. En ensamblador, se usan directivas como DB, DW, DD, DQ, y DT para asignar y inicializar espacios de almacenamiento. Ejemplos:

- DB: Define 1 byte.
- DW: Define 2 bytes.
- DD: Define 4 bytes.

Para datos no inicializados, se usan directivas como RESB, RESW, y RESD, que reservan espacio sin asignar valores específicos.

### **Ejemplo de inicialización:**

*value DW 25159* ;Inicializa a 25159 (6247H)

Además, las definiciones pueden usar expresiones matemáticas para clarificar el código, como *max\_marks DW 7\*25* ;en lugar de DW 175.

Esto facilita la comprensión del programa.

## **II. Lenguaje ensamblador**

**Multiple Initializations (Múltiples Inicializaciones):** En lugar de definir un arreglo elemento por elemento, la directiva TIMES permite inicializar varios elementos al mismo valor. Es útil para definir arrays o tablas de manera más eficiente. Por ejemplo,

*marks TIMES 8 DW 0* ;inicializa 8 elementos del array marks con el valor 0.

**Symbol Table (Tabla de Símbolos):** Cuando se define un espacio de almacenamiento con nombres simbólicos, el ensamblador asigna un valor de desplazamiento a cada nombre durante el proceso de ensamblado. El desplazamiento de un símbolo se basa en la cantidad de bytes que ocupan las variables precedentes. La tabla de símbolos guarda estos valores, facilitando la ubicación de los datos en la memoria.

**Where Are the Operands (Ubicación de los Operandos):** Las instrucciones en lenguaje ensamblador requieren operandos, que pueden estar en registros del procesador, en la instrucción misma, en la memoria principal o en un puerto de E/S. Los diferentes modos de direccionamiento determinan cómo se especifican estos operandos, dependiendo de si están en registros, en la memoria o dentro de la propia instrucción.

**Register Addressing Mode (Modo de Direccionamiento por Registro):** Los operandos están en los registros internos del procesador. Este modo es eficiente porque no requiere acceso a la memoria. Ejemplo:

*mov EAX,EBX* ;copia el valor del registro EBX en el registro EAX.

**Immediate Addressing Mode (Modo de Direccionamiento Inmediato):** El dato se especifica directamente en la instrucción, no en la memoria. Se utiliza típicamente para el operando fuente, como en

*mov AL,75* ;donde el valor 75 está en la instrucción y el destino es el registro AL.

**Direct Addressing Mode (Modo de Direccionamiento Directo):** El operando está en la memoria y su dirección de desplazamiento se especifica directamente en la instrucción. Ejemplo:

*mov AL,[response]* ;copia el valor almacenado en response al registro AL.

**Indirect Addressing Mode (Modo de Direccionamiento Indirecto):** Este modo es útil para trabajar con estructuras de datos complejas, como arrays. En lugar de especificar la dirección directamente, se utiliza un registro (como EBX) que apunta a la dirección de los datos en la memoria. Ejemplo:

*mov [EBX],100* ;almacena el valor 100 en la dirección apuntada por EBX.

**LEA Instruction (Instrucción LEA):** La instrucción lea calcula direcciones efectivas en tiempo de ejecución. Por ejemplo,

*lea EBX,[table]*

calcula la dirección del primer elemento de table y la carga en el registro EBX. Ofrece más flexibilidad que mov, que resuelve las direcciones en tiempo de ensamblado.

Estos puntos destacan las partes más importantes de cada sección para ayudarte a estudiar de manera más eficiente.

### III. Instrucciones de lenguaje ensamblador

1. **Instrucción MOV:** La instrucción mov transfiere datos entre dos operandos. Los datos se mueven de la fuente al destino sin alterar la fuente. Esta instrucción puede transferir datos entre registros, memoria e inmediatos, pero no permite transferencias directas de memoria a memoria. Para clarificar ambigüedades al mover valores a memoria, se usan especificadores de tipo como BYTE y WORD para definir el tamaño del dato que se está moviendo.
2. **Instrucciones de aritmética simples:**
  - **INC y DEC:** inc incrementa un operando en 1, y dec lo decrementa en 1. Ambos operan sobre registros o memoria, pero no sobre valores inmediatos.
  - **ADD:** add suma dos operandos (registros, memoria o inmediatos), modificando el destino. Para sumar 1, se prefiere usar inc para mejorar la legibilidad y ahorrar espacio.

- **SUB y CMP:** sub resta dos operandos, modificando el destino. cmp compara dos operandos sin modificar los valores, preparando la siguiente instrucción condicional.

### 3. Ejecución condicional:

- **Salto incondicional:** jmp transfiere el control del programa a una etiqueta específica, sin evaluar ninguna condición.
- **Salto condicional:** Instrucciones como je, jg, jl, etc., transfieren el control dependiendo de la comparación anterior. Por ejemplo, je salta si los operandos son iguales. Estas instrucciones dependen de los valores de las banderas (flags) establecidas previamente en operaciones como cmp.

4. **Instrucciones de iteración:** Los bucles pueden implementarse con instrucciones de salto, pero la instrucción loop simplifica el proceso, utilizando el registro ECX como contador de iteraciones. Cada vez que se ejecuta loop, ECX se decrementa y el control se transfiere al objetivo si no ha llegado a cero.

5. **Instrucciones lógicas:** Las operaciones lógicas (and, or, xor y not) se aplican de manera bit a bit. Estas instrucciones pueden establecer algunas banderas, permitiendo su uso en estructuras condicionales.

| statement_1                              | AL  | BL  | Action taken                             |
|------------------------------------------|-----|-----|------------------------------------------|
| je go_back                               | 56H | 56H | Program control is transferred to inc AL |
| jg go_back                               | 56H | 55H | Program control is transferred to inc AL |
| jg go_back<br>jl go_back                 | 56H | 56H | No jump; executes<br>mov BL, 77H         |
| jle go_back<br>jge go_back               | 56H | 56H | Program control is transferred to inc AL |
| jne go_back<br>jg go_back<br>jge go_back | 27H | 26H | Program control is transferred to inc AL |

**Figura 1.** Algunos ejemplos de saltos condicionales

| and operation |                   |                   |
|---------------|-------------------|-------------------|
| Input bits    |                   | Output bit        |
| source $b_i$  | destination $b_i$ | destination $b_i$ |
| 0             | 0                 | 0                 |
| 0             | 1                 | 0                 |
| 1             | 0                 | 0                 |
| 1             | 1                 | 1                 |

| or operation |                   |                   |
|--------------|-------------------|-------------------|
| Input bits   |                   | Output bit        |
| source $b_i$ | destination $b_i$ | destination $b_i$ |
| 0            | 0                 | 0                 |
| 0            | 1                 | 1                 |
| 1            | 0                 | 1                 |
| 1            | 1                 | 1                 |

| xor operation |                   |                   |
|---------------|-------------------|-------------------|
| Input bits    |                   | Output bit        |
| source $b_i$  | destination $b_i$ | destination $b_i$ |
| 0             | 0                 | 0                 |
| 0             | 1                 | 1                 |
| 1             | 0                 | 1                 |
| 1             | 1                 | 0                 |

**Figura 2.** Tabla de verdad de las operaciones lógicas